MOBILE

# Mobile App Architecture Visual Guide

Native vs cross-platform, lifecycles,
offline-first sync, and push notifications.

**ANIMATED WALKTHROUGH**
Step through pages to play the animation

STEP 1/6

# Native vs Cross-Platform

One codebase or two — the founding tradeoff.

**NATIVE**

Swift/Kotlin — max perf, full APIs, 2 codebases

**CROSS-PLATFORM**

Flutter/RN — 1 codebase, near-native UI, some gaps

**DECIDE BY**

platform-specific hardware/UX depth vs team size and speed

Cross-platform wins on velocity for CRUD apps; native wins for games, AR, and deep OS integration.

STEP 2/6

# App Lifecycles

The OS can kill your app anytime — design for interruption.

Save state on backgrounding. Restore on launch. Never assume you will be resumed.

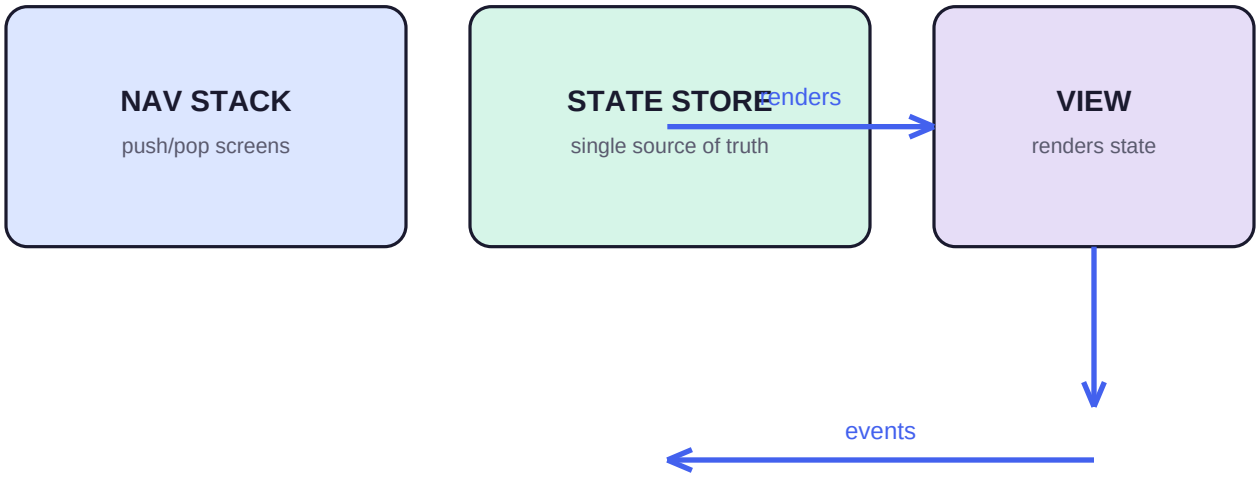
**iOS vs ANDROID**

suspended vs cached processes — same lesson, different names

STEP 3/6

# UI Architecture: Navigation & State

Screens are a stack; state flows down, events flow up.

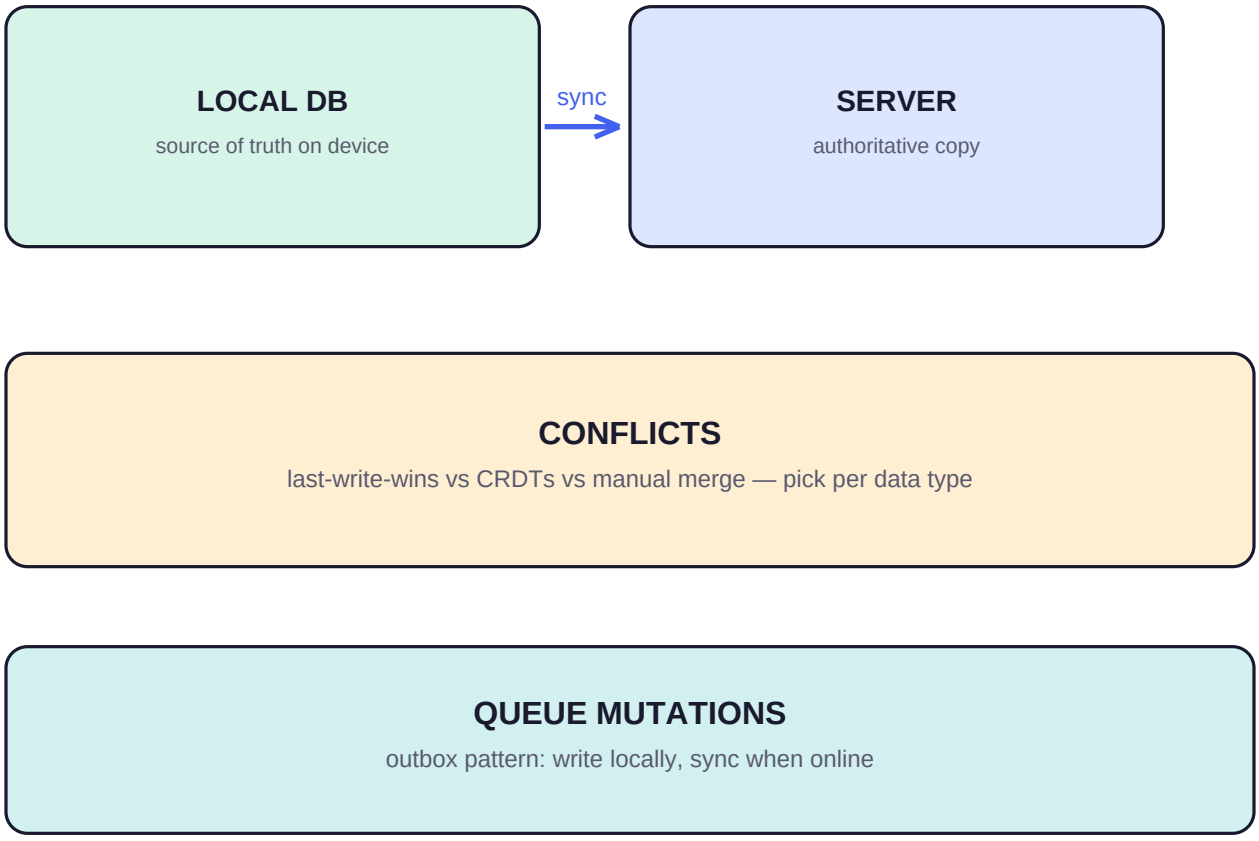



Unidirectional data flow keeps complex screens debuggable — the pattern behind Redux, Bloc, and friends.

STEP 4/6

# Offline-First Sync

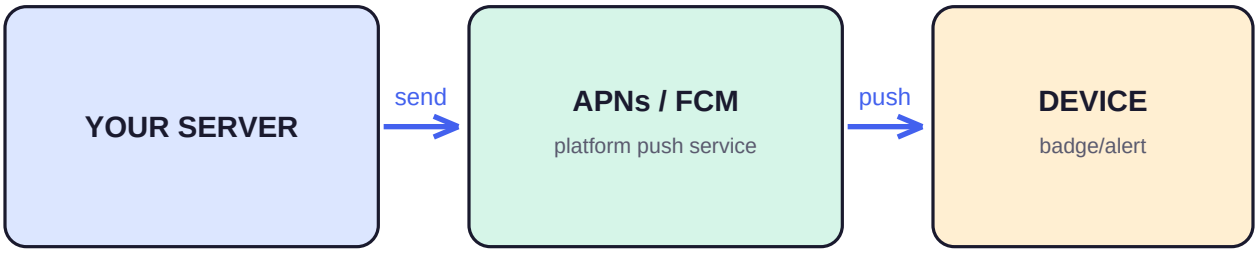
Treat the network as an optimization, not a requirement.



STEP 5/6

# Push Notifications

Your server -> platform service -> device — you never reach the phone directly.




**PERMISSION**

users opt in — one spammy blast and you are muted forever

Silent pushes wake the app for background sync; visible pushes must earn attention.

STEP 6/6

# Distribution & Release

Stores gate releases — plan the pipeline.

| | |
|---|---|
| **Beta channels** | TestFlight / Play internal tracks — real devices before launch |
| **Staged rollouts** | 1% -> 10% -> 100%; halt on crash spikes |
| **Code signing** | certificates + provisioning — lose the key, lose the app |
| **OTA updates** | JS/bundled assets can update instantly; native code needs review |

STEP ✓/✓

# Key Takeaways

## Mobile Architecture — Remember

- Native maximizes platform depth; cross-platform maximizes team velocity — choose deliberately.
- The OS can kill your app anytime: save on background, restore on launch.
- Unidirectional data flow (state down, events up) keeps complex UIs debuggable.
- Offline-first: local DB as truth, outbox queue for mutations, explicit conflict policy.
- Push flows server -> APNs/FCM -> device; permission is precious — never spam.
- Ship via beta tracks and staged rollouts; guard your signing keys.

**Keep exploring**

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev